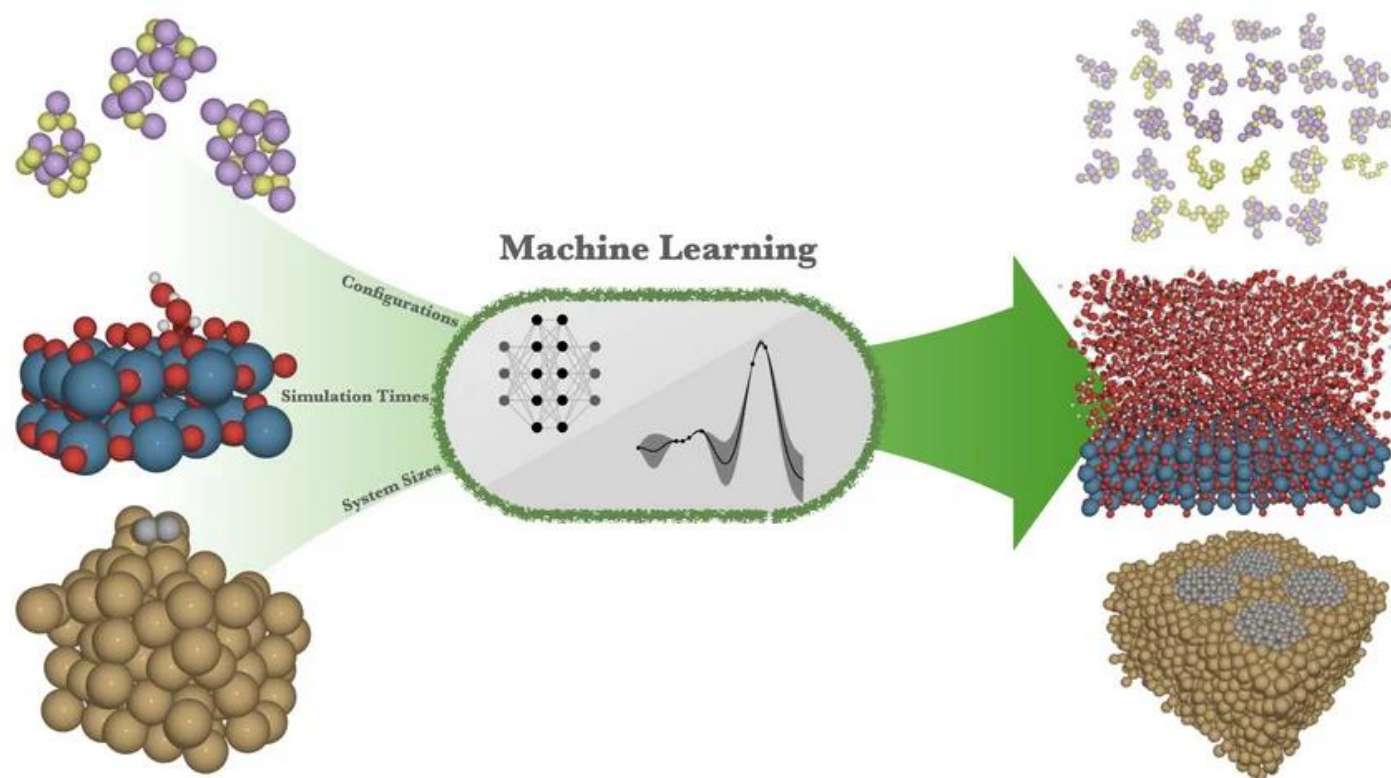
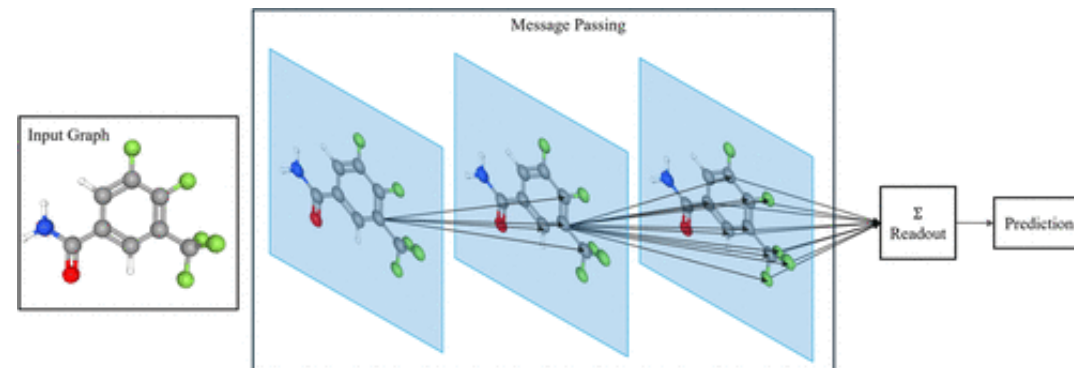


Graph Neural Networks and Machine Learning Interatomic Potentials



Lecture by Madeline Murphy

Images from:
https://www.fhi.mpg.de/739215/heenen_group
J. Chem. Inf. Model. 2026, 66, 3, 1316–1336

Central goal of today

How can we get DFT-quality calculations and molecular dynamics at useful time and length scales?

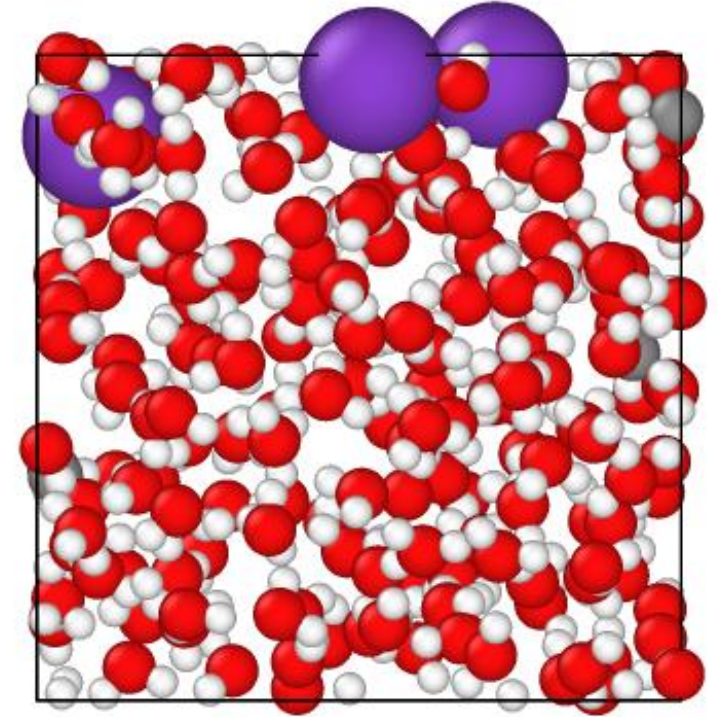
Agenda

1. **What are MLIPs and why are they useful?**
2. Core concepts of MLIPs: representations, descriptors, architectures
3. Basics of Graph Neural Networks
4. Foundation models
5. Data generation and training from scratch / finetuning
6. General considerations / best practices
7. The future of MLIPs

What is Molecular Dynamics?

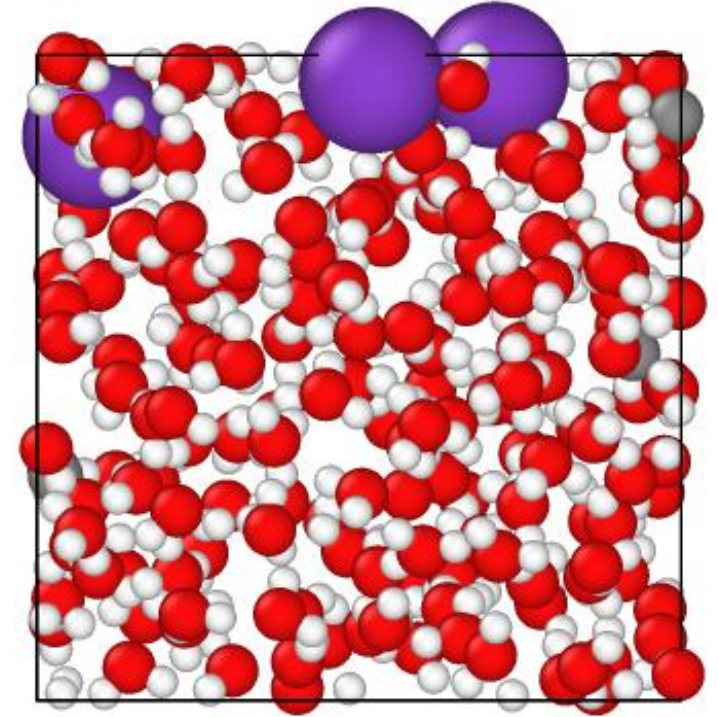
- Creates a 'stop-motion' movie of molecules in action
- Classical physics describes the propagation of time
 - Force field describes the energy landscape
 - Newton's Second Law propagates motion forward in time in accordance with the computed forces
 - Updates positions and velocities by a tiny time step (1-2 femtoseconds) and repeats

$$F = -\nabla U = ma$$



Benefit of Dynamics over a Static Model

- **Conformational dynamics:** watch molecules move, open/close, and uncoil
- **Explicit solvation:** watch the explicit solvent surround the molecule of interest and rearrange over time
- **Thermodynamics/kinetics:** statistical averages across the trajectory give relevant thermodynamic (free energy differences, enthalpy, entropy) and kinetic properties (diffusion rates, ligand unbinding pathways)
- **Important point:** standard MD with empirical force fields is non-reactive



What is Molecular Dynamics?

The central engine of MD is the force computed at each time step:

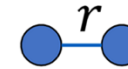
$$F = -\nabla U = ma$$

Where do the forces come from?

- Empirical Forcefields
- Density Functional Theory

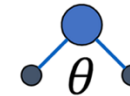
$$U_{total} = \underbrace{U_{bond} + U_{angle} + U_{Pdihe} + U_{Idihe}}_{\text{Bonded interactions}} + \underbrace{U_{LJ} + U_{PC}}_{\text{Non-Bonded interactions}}$$

Bond potential:
Spring



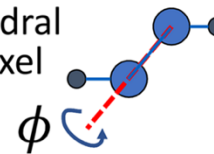
U_{bond}

Angle potential:
Hinge



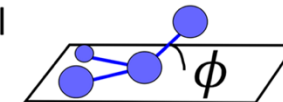
U_{angle}

Proper dihedral potential: Axle



U_{Pdihe}

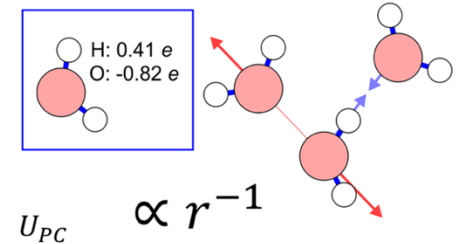
Improper dihedral potential



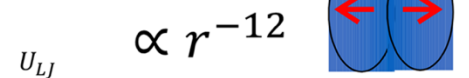
U_{Idihe}

Electrostatic potential

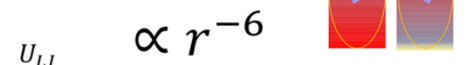
Bonding + Electronegativity:
Partial charges on atoms



Repulsive Pauli
Exclusion principle



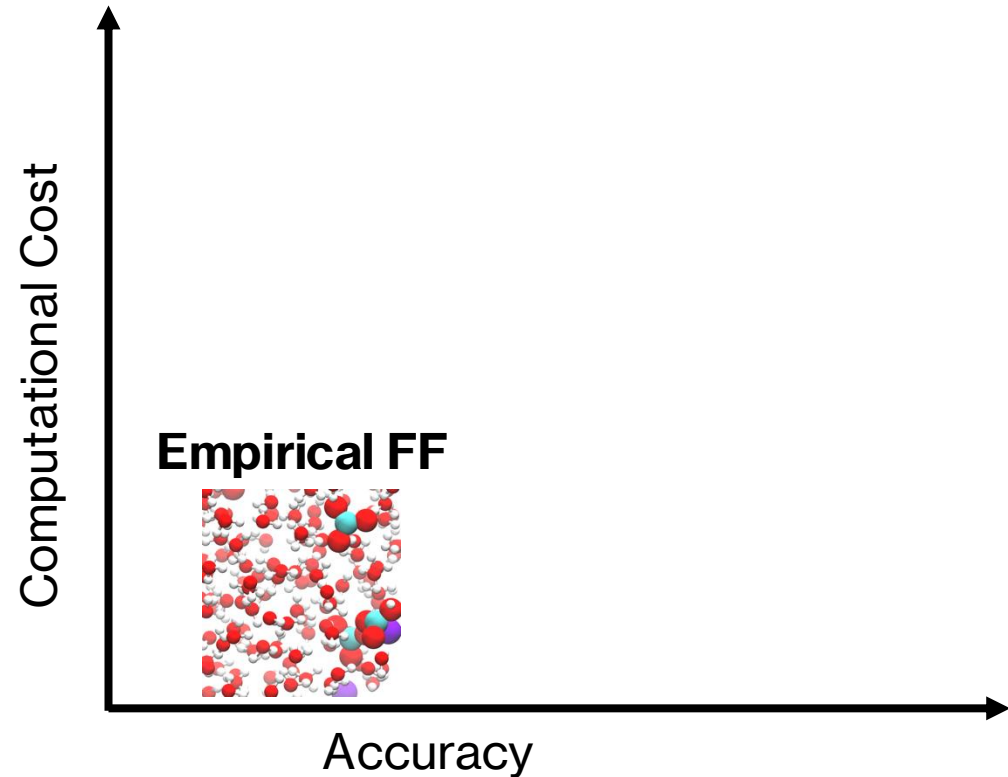
Attractive
Van-der-Waals



Bunker, A. & Rog, T., *Front. Mol. Biosci.*, **2020**.

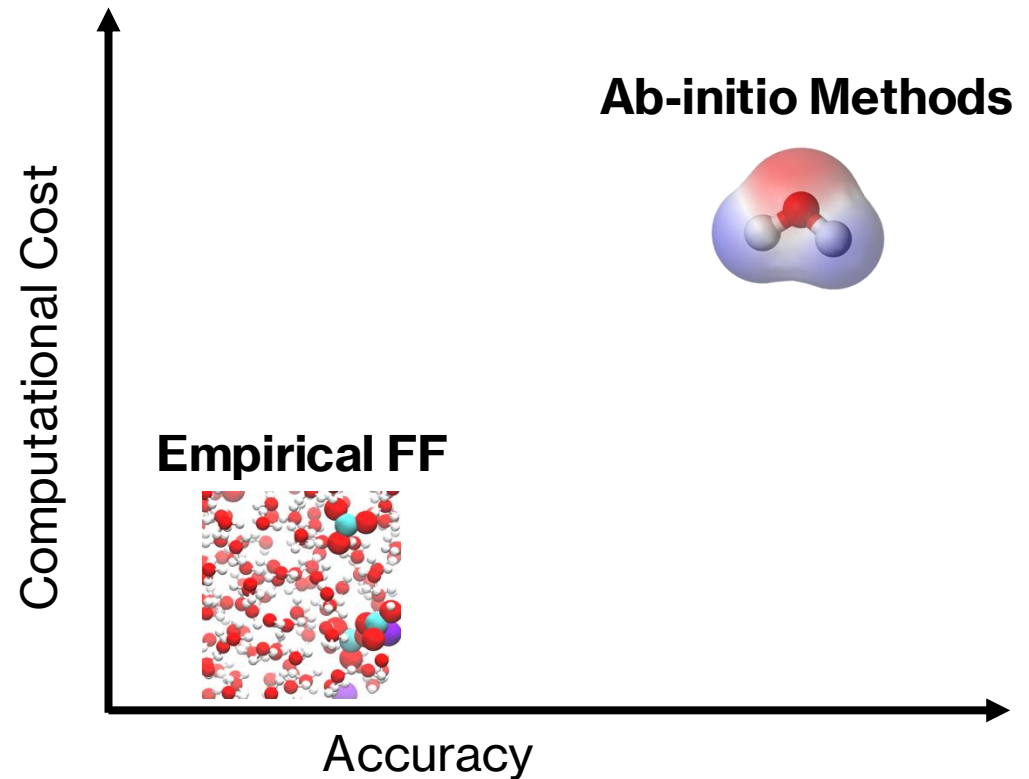
The Classical Dilemma: Accuracy vs. Speed Tradeoff

- Parameterized description of simplistic models for interparticle interactions
- Fast and cheap $\sim O(N)$ or $\sim O(N \ln N)$
- Fixed chemistry, lacks reactive transferability



The Classical Dilemma: Accuracy vs. Speed Tradeoff

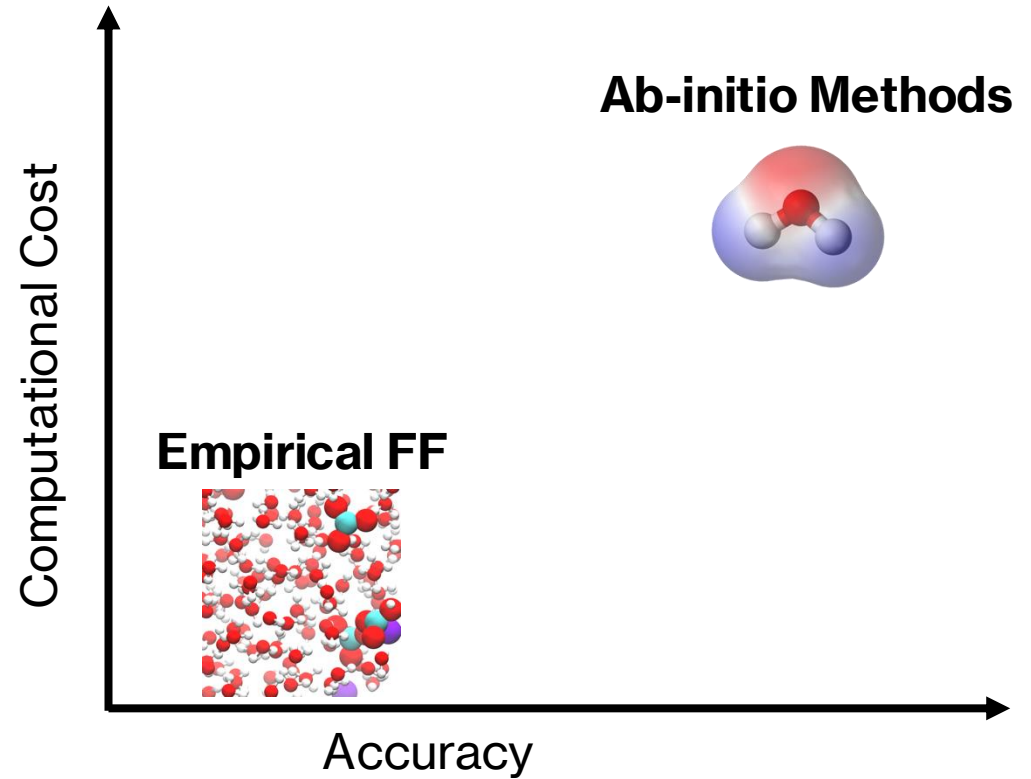
- *Ab initio* calculations (DFT, couple cluster, etc.) demonstrate high chemical accuracy
- Scales poorly with system size
 $\sim O(N^3)$
- Limited to small systems and short times



The Classical Dilemma: Accuracy vs. Speed Tradeoff

Empirical FF: Fast but inaccurate

Ab-Initio Methods (DFT):
accurate but too slow

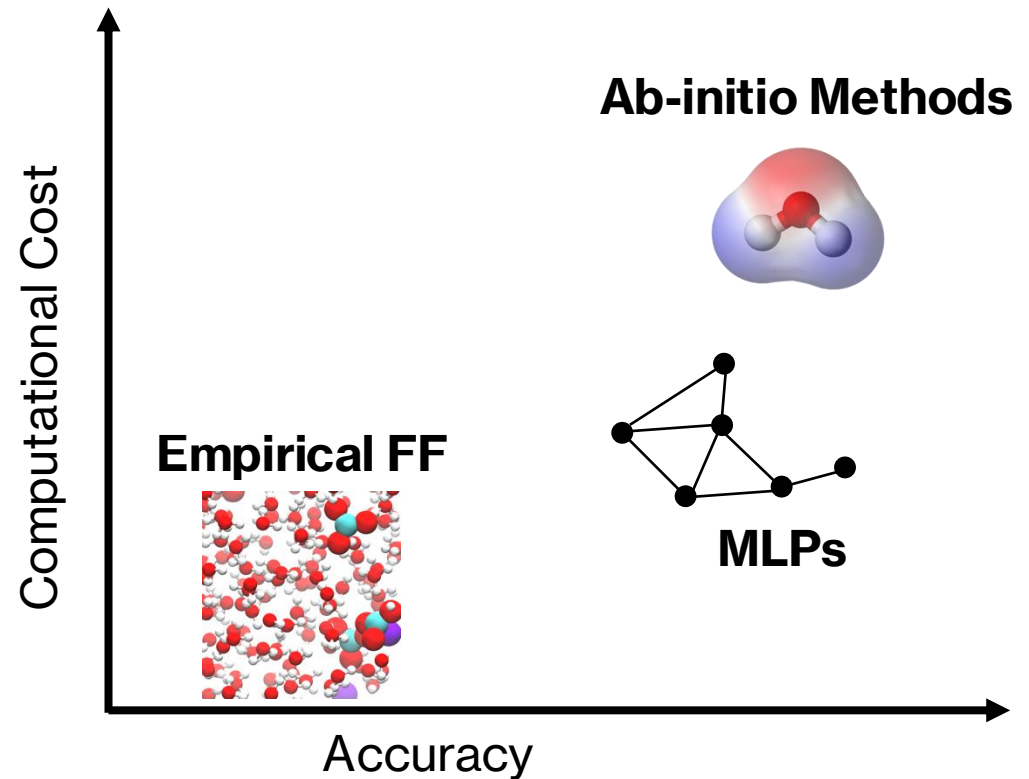


The Classical Dilemma: Accuracy vs. Speed Tradeoff

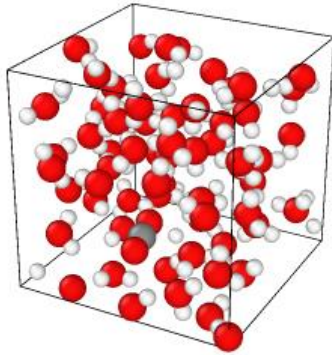
- MLIPs promise near-DFT accuracy at classical, empirical force field speed
- Train machine learning models on quantum mechanical data to predict the energy and forces of atomic configurations

$$E = f(Z_1, r_1, Z_2, r_2, \dots, Z_N, r_N)$$

- Trains on local atomic environments to observe linear scaling $\sim O(N)$



What is the Machine Learning Problem?

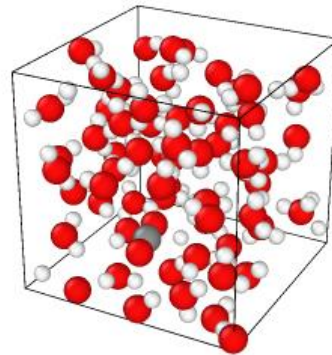


DFT



Energy, Forces

...train the model on DFT data



MLIP

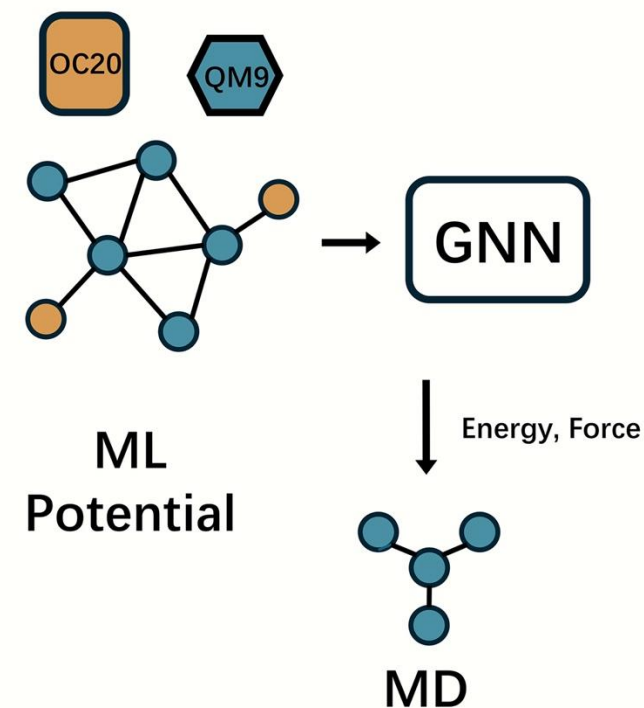


Energy, Forces

The ML model isn't learning chemistry from scratch, it's learning to imitate the quantum mechanical calculator

Machine Learning Interatomic Potentials

- An MLIP is a surrogate model to approximate the potential energy surface (PES) of your system
- **Input:** Atomic positions (R) and species (Z)
- **Output:** Total Energy and Atomic Forces
- **Forces:** Calculated as analytical gradients $F_i = -\nabla E_{tot}$

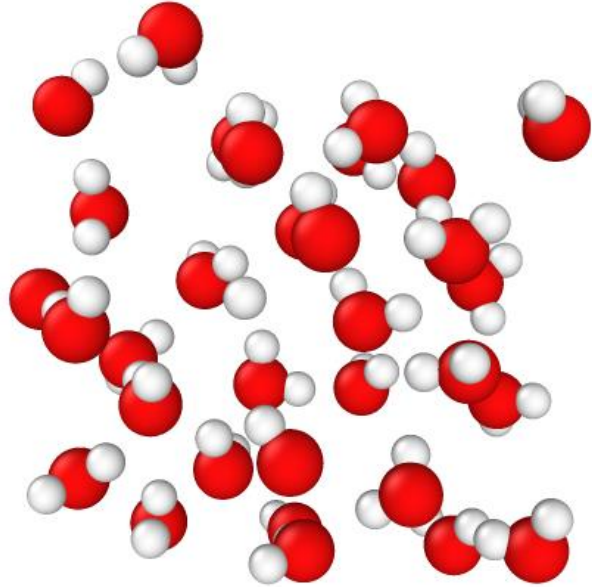


Agenda

1. What are MLIPs and why are they useful?
- 2. Core concepts of MLIPs: representations, descriptors, architectures**
3. Basics of Graph Neural Networks
4. Foundation models
5. Data generation and training from scratch / finetuning
6. General considerations / best practices
7. The future of MLIPs

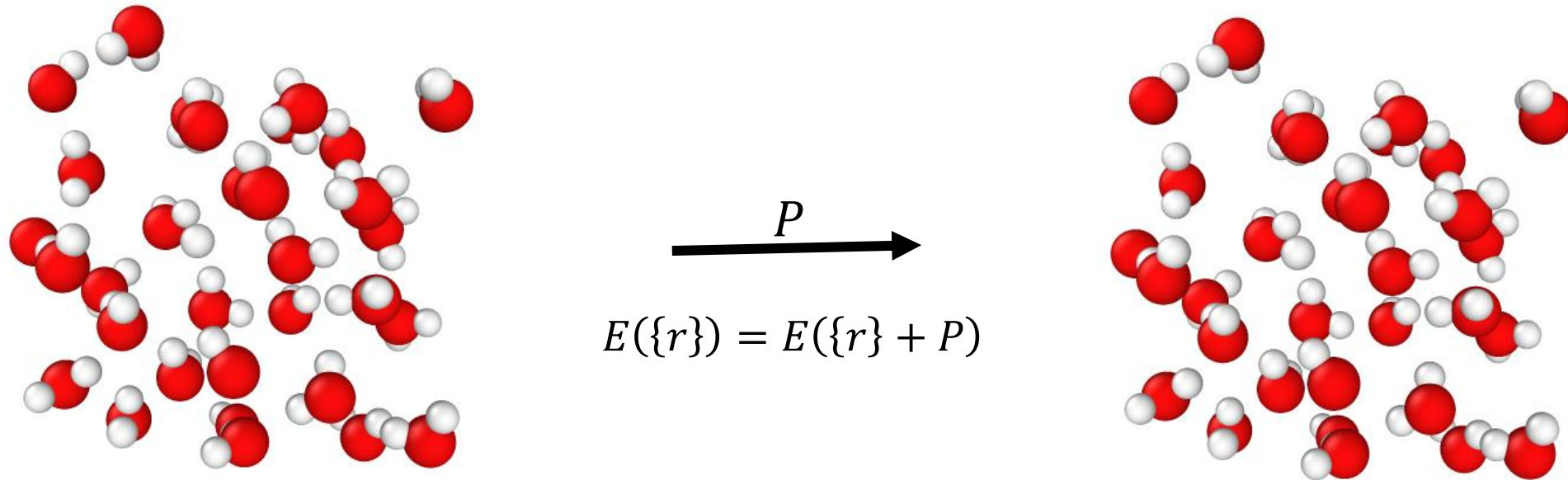
Describing a Molecular System

What is our system? And what do we know to be true about this system?



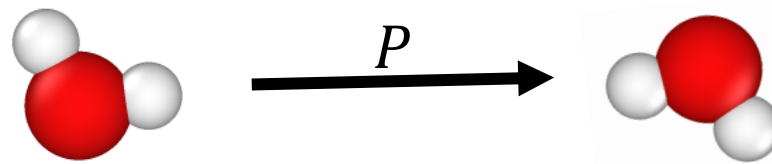
Symmetry and Physical Invariants

- Translation invariance: moving the entire space doesn't change the energy



Symmetry and Physical Invariants

- Translation invariance: moving the entire space doesn't change the energy
- Rotational invariance/equivariance: rotating the system preserves energy (invariant) and rotates forces (equivariant)

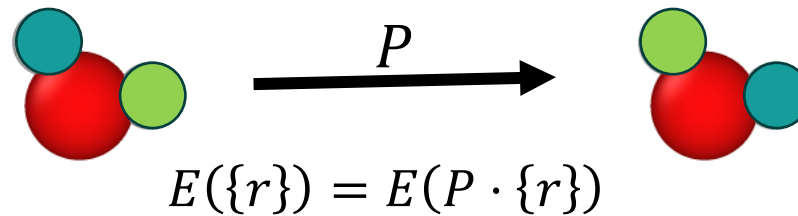


$$E(\{r\}) = E(P \cdot \{r\})$$

$$F(P \cdot \{r\}) = PF(\{r\})$$

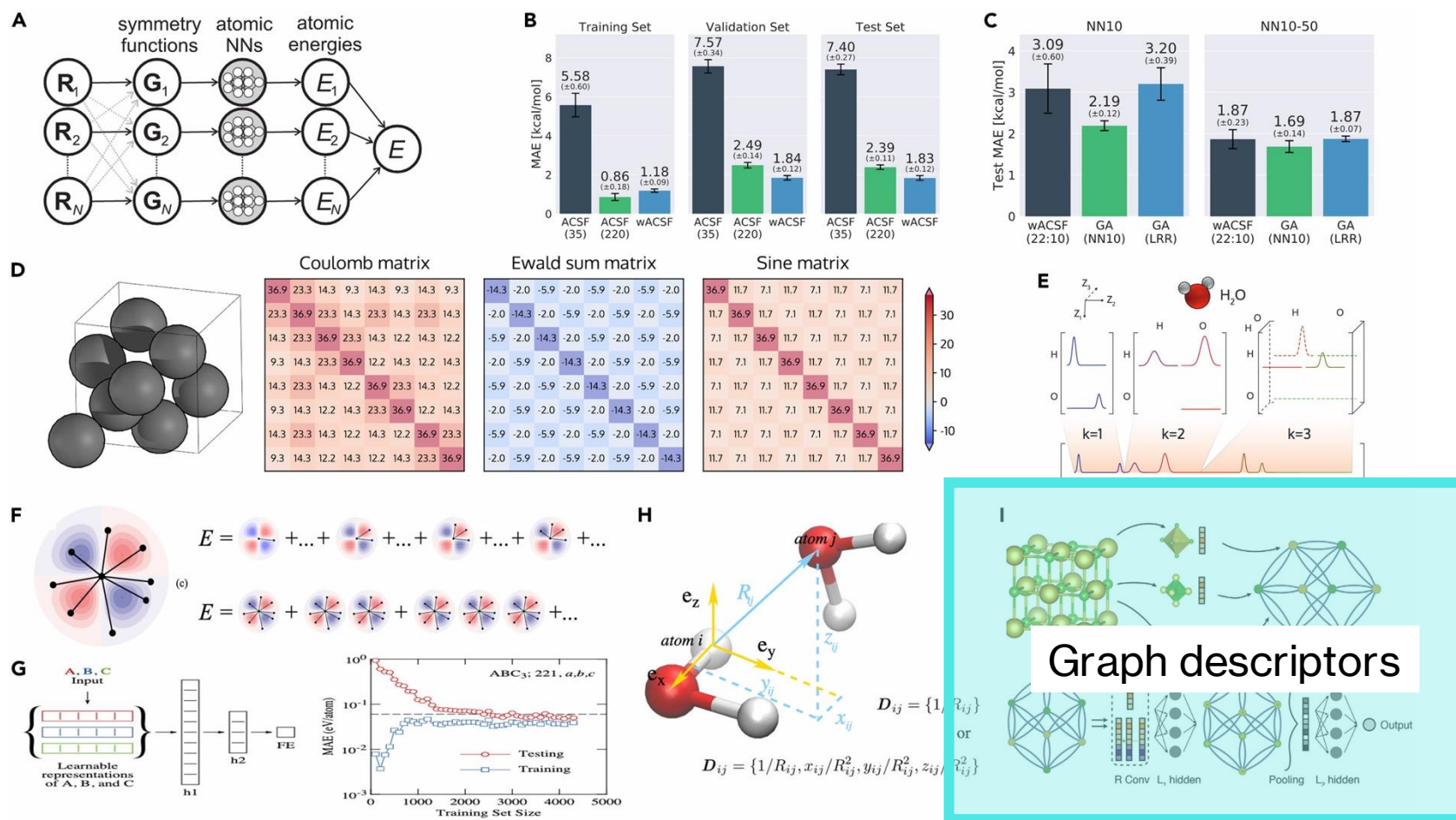
Symmetry and Physical Invariants

- Translation invariance: moving the entire space doesn't change the energy
- Rotational invariance/equivariance: rotating the system preserves energy (invariant) and rotates forces (equivariant)
- Permutational invariance: swapping the index of two identical atoms preserves energy



Molecular Representations

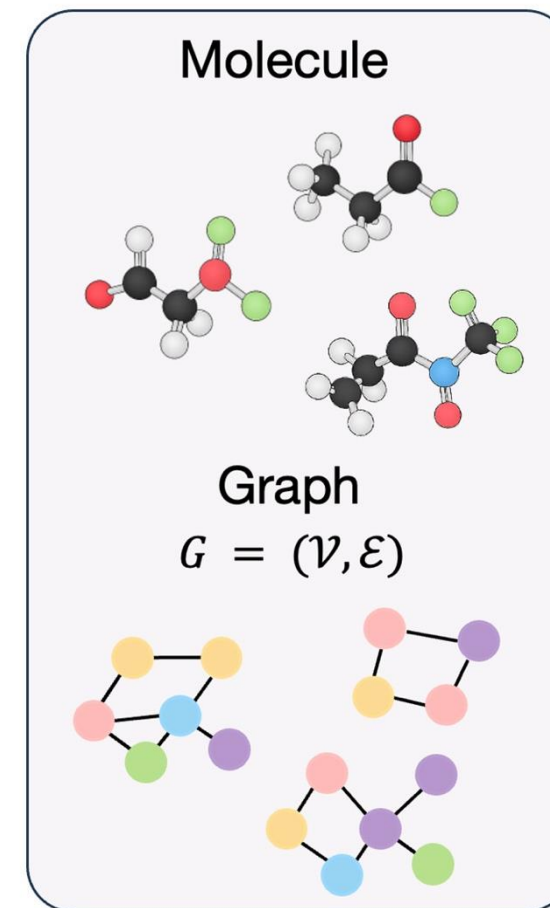
A naïve neural network would think all these permutations are different molecules



Molecular Representations

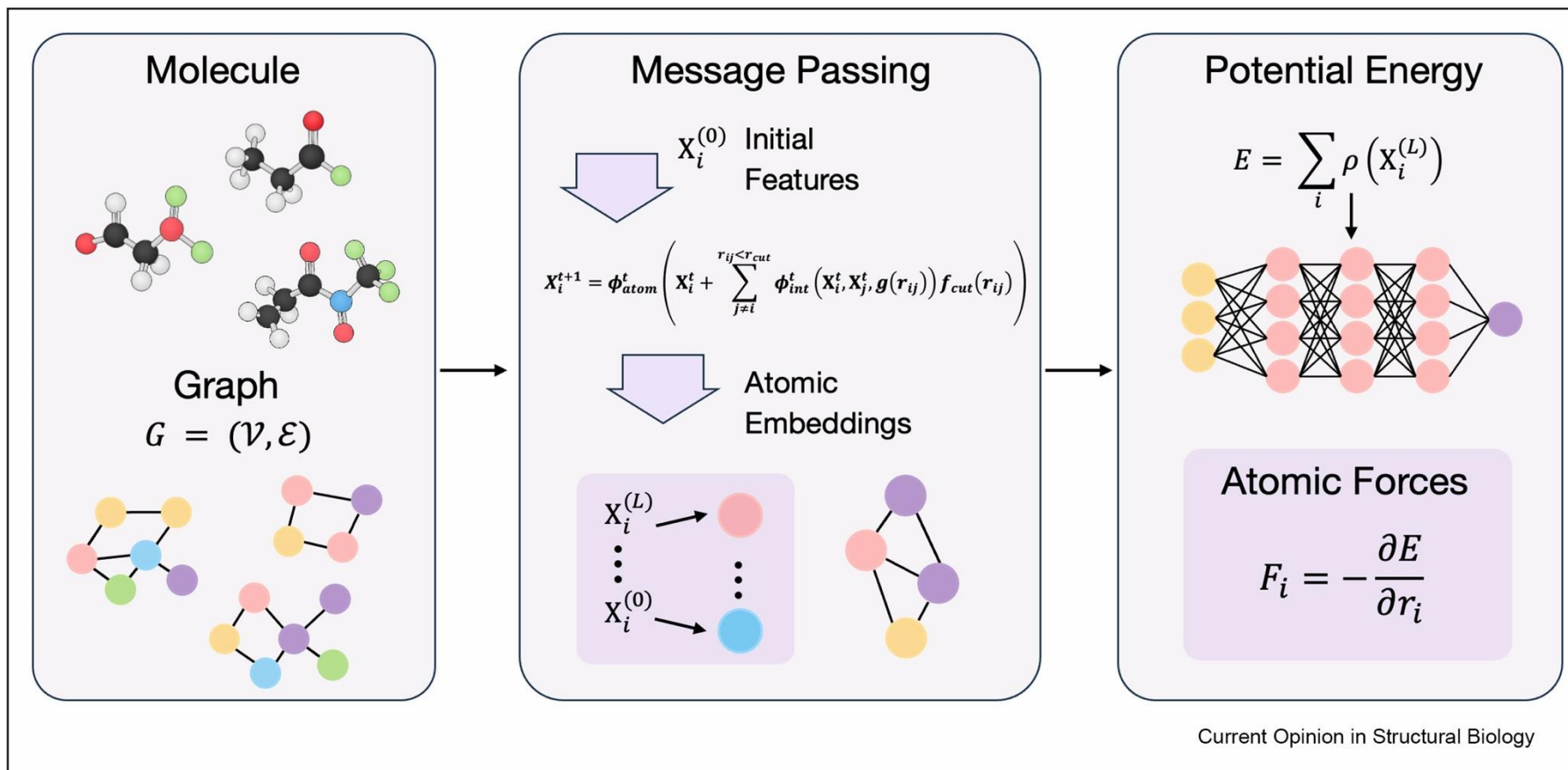
Learned Representation

- Representations must remove these ambiguities
- Learned descriptors (SchNet, NequIP, MACE)
- Once symmetry-preserving representation is known, the problem can be formatted as a graph



Core Concepts of MLIPs

Architecture Overview: Mapping inputs to Energies



Agenda

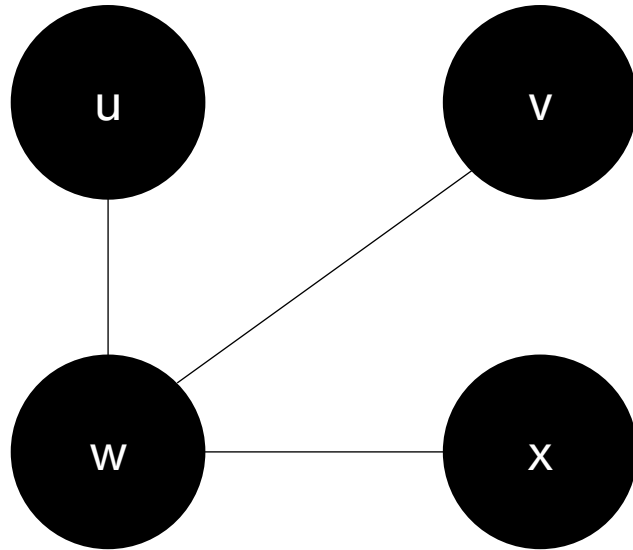
1. What are MLIPs and why are they useful?
2. Core concepts of MLIPs: representations, descriptors, architectures
- 3. Basics of Graph Neural Networks**
4. Foundation models
5. Data generation and training from scratch / finetuning
6. General considerations / best practices
7. The future of MLIPs

Basics of Graph Neural Networks

Graph (G) is defined by the following components:

Nodes (V) : the set of nodes

Edges (E): set of tuples of form (u, v) , where there is an edge between node u and v



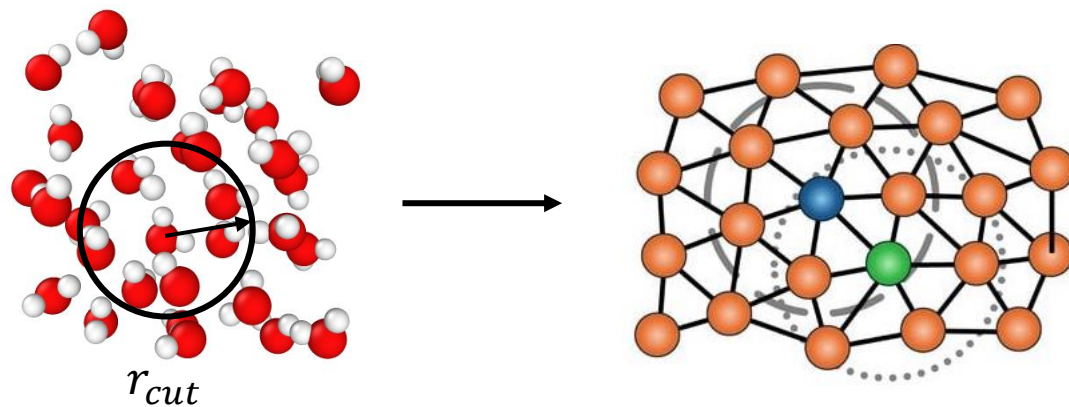
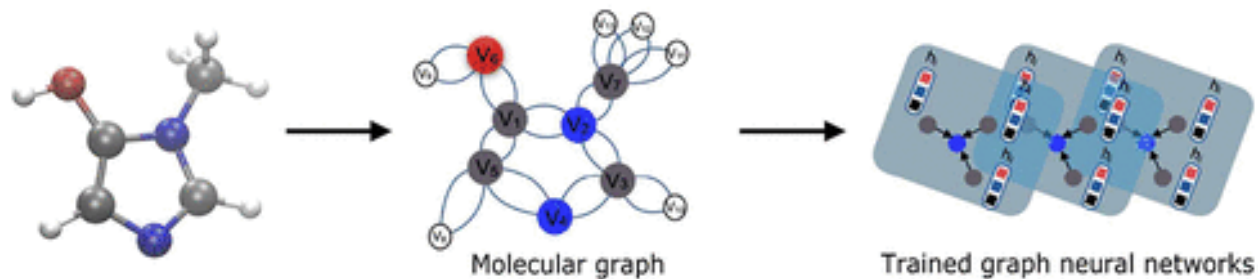
$$G = (V, E)$$

$$V = \{u, v, w, x\}$$

$$E = \{(u, w), (v, w), (w, x)\}$$

Basics of Graph Neural Networks

Mapping the molecule to a graph



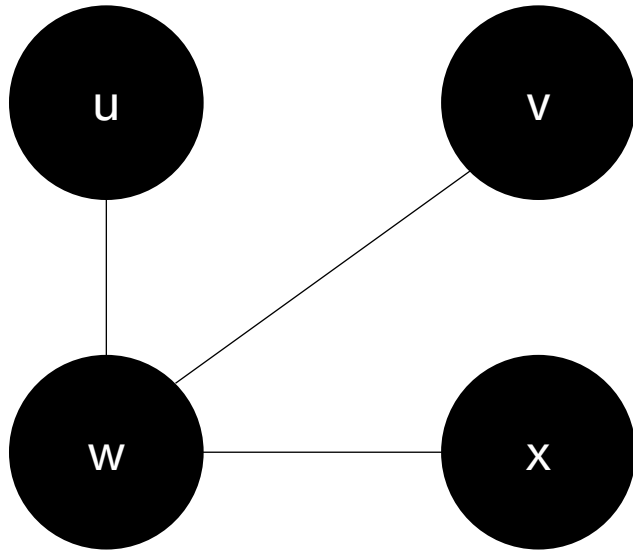
Nodes: atoms, initial feature vector based on identity

Edges: connect atoms within the cutoff distance of one another, stores the distance r_{ij}

Basics of Graph Neural Networks

Graph encoding to an Adjacency Matrix: $A \in \mathbb{R}^{|V| \times |V|}$

- Rows/columns number the nodes, matrix elements encode the edges



$A =$

u	v	w	x	
0	0	1	0	u
0	0	1	0	v
1	1	0	1	w
0	0	1	0	x

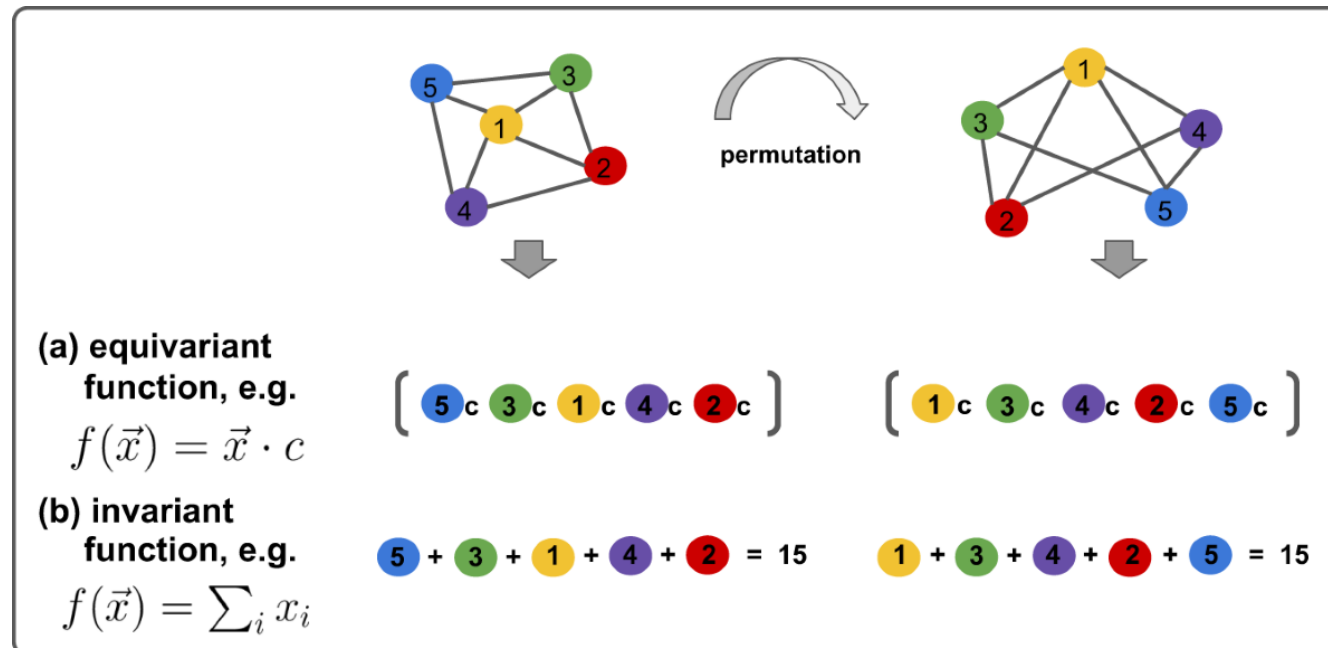
Basics of Graph Neural Networks

General considerations for GNN:

- Architecture is chosen to be permutation invariant and equivariant

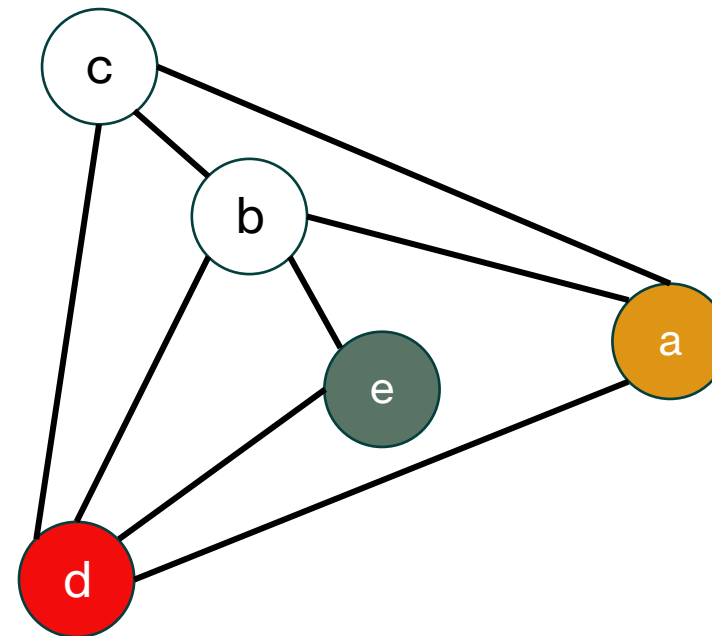
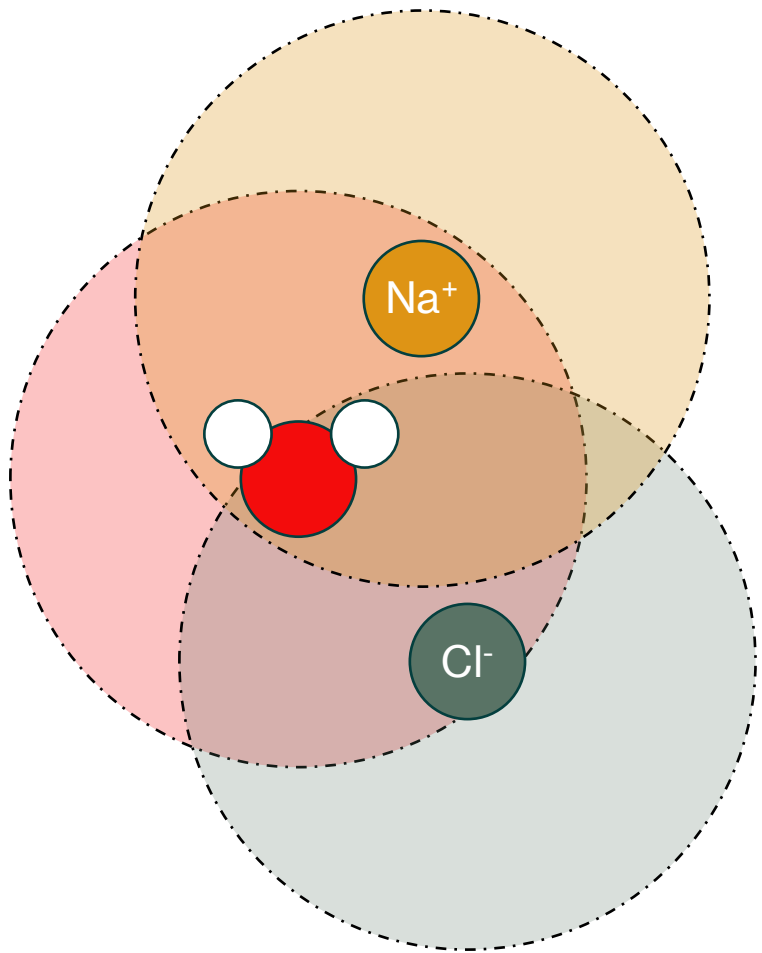
Permutation Invariance property: $f(\mathbf{PAP}^T) = f(\mathbf{A})$

Permutation Equivariance property: $f(\mathbf{PAP}^T) = \mathbf{P} f(\mathbf{A})$



Basics of Graph Neural Networks

Neural Message Passing



Node Features: Atom Type

$$X = \begin{bmatrix} a \\ b \\ c \\ d \\ e \end{bmatrix} = \begin{bmatrix} 11 \\ 1 \\ 1 \\ 8 \\ 17 \end{bmatrix}$$

Edge Features: Atomic distances

ex. (a, b) = 4 Å

Basics of Graph Neural Networks

Neural Message Passing

Goal: Combine the information from the neighboring nodes to encode graph information

$$h_u^{(k+1)} = \text{UPDATE}^{(k)} \left(h_u^{(k)}, \text{AGGREGATE}^{(k)} \left(\{h_v^{(k)}, \forall v \in N(u)\} \right) \right)$$

$$h_u^{(k+1)} = \text{UPDATE}^{(k)} \left(h_u^{(k)}, m_{N(u)}^{(k)} \right)$$

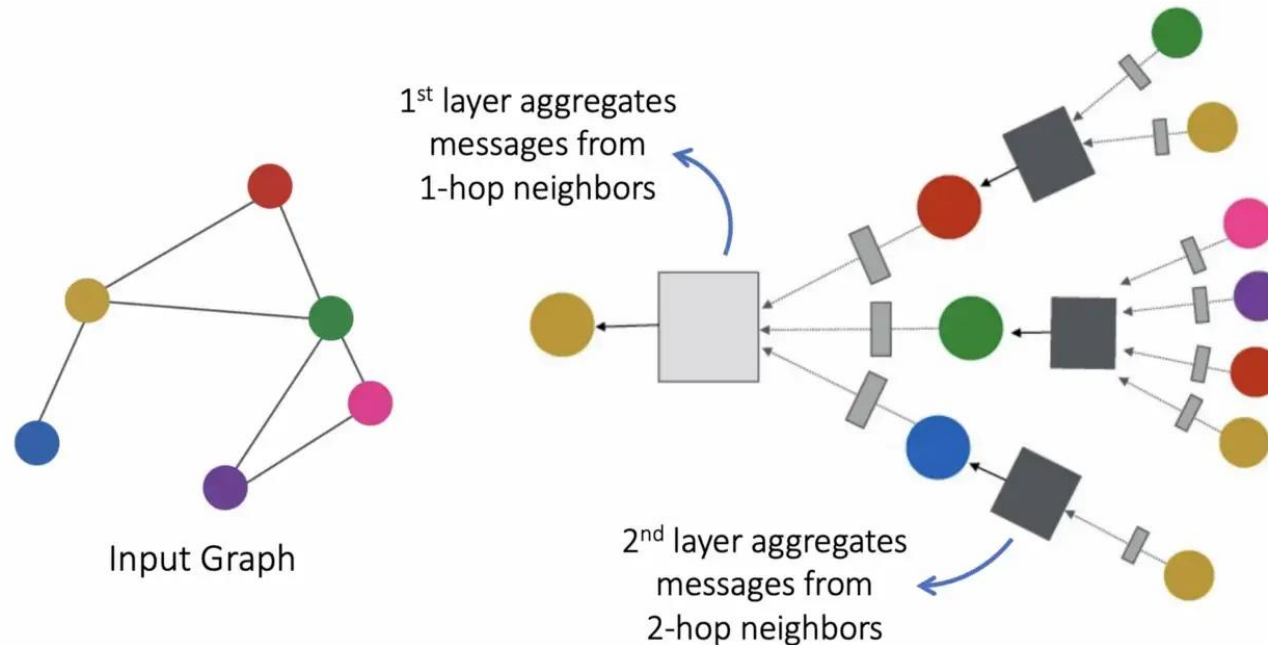
For each layer, k , :

- Aggregate all embeddings from u 's neighbors to generate a message $m_{N(u)}^{(k)}$
- Update the embedding $h_u^{(k)}$ of node u by combining information from the previous embedding $h_u^{(k-1)}$ with the message $m_{N(u)}^{(k)}$

Basics of Graph Neural Networks

Neural Message Passing

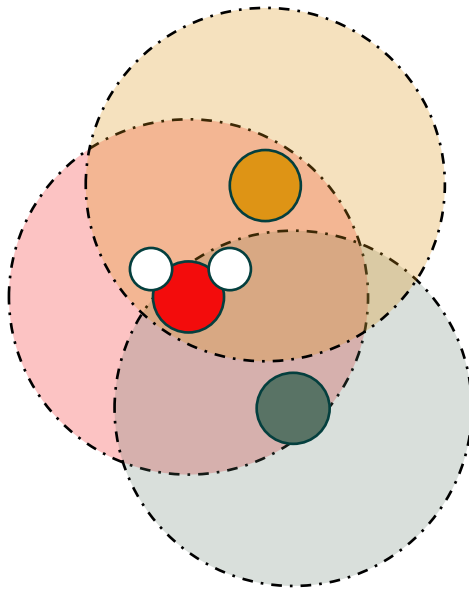
Goal: Combine the information from the neighboring nodes to encode graph information



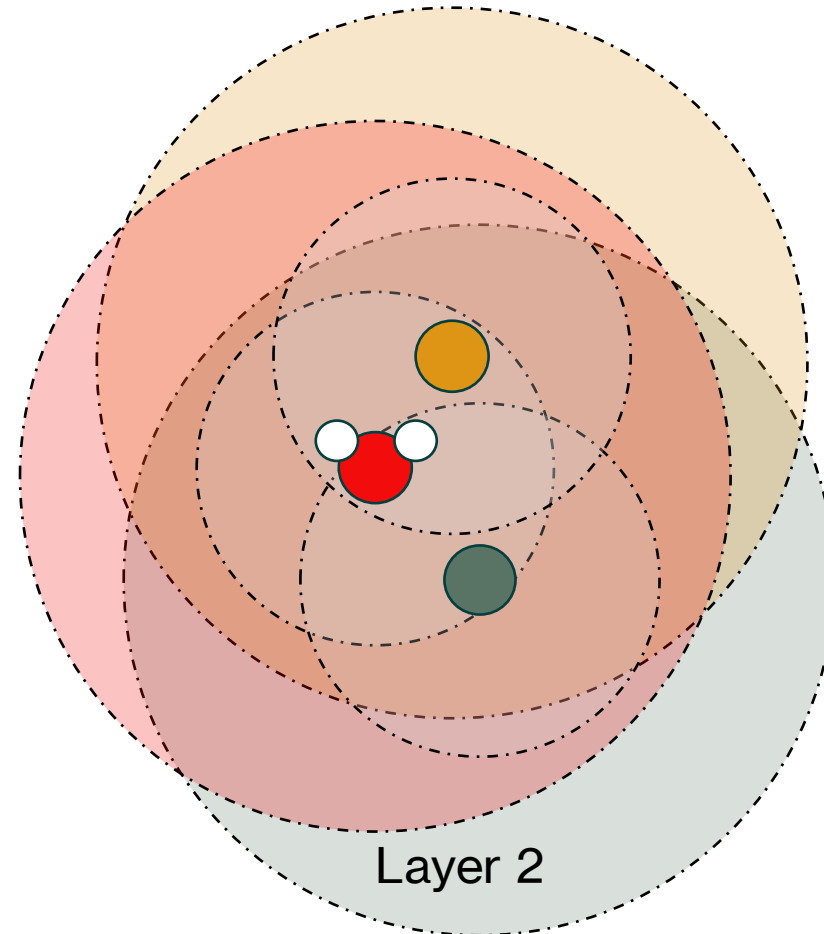
Basics of Graph Neural Networks

Neural Message Passing

Multi-layered GNNs extend the effective radius out past the cutoff radius



Layer 1



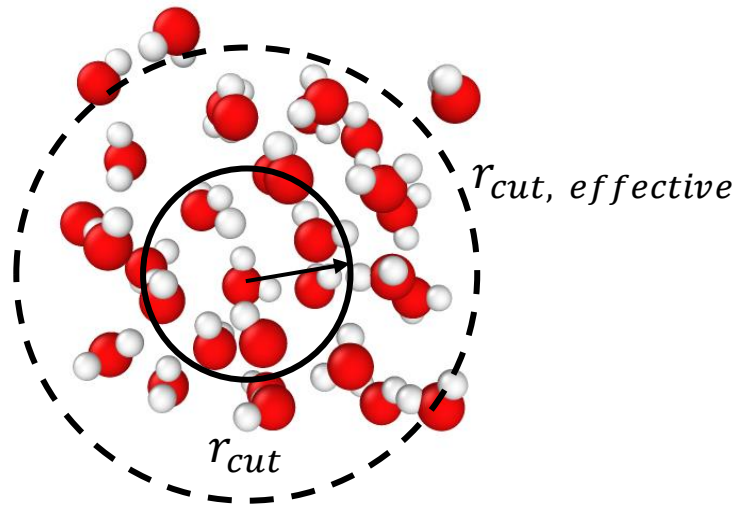
Layer 2

Basics of Graph Neural Networks

Neural Message Passing: how atoms feel other atoms beyond their immediate neighbors

Multi-hop effect: information propagates one shell per layer of the model

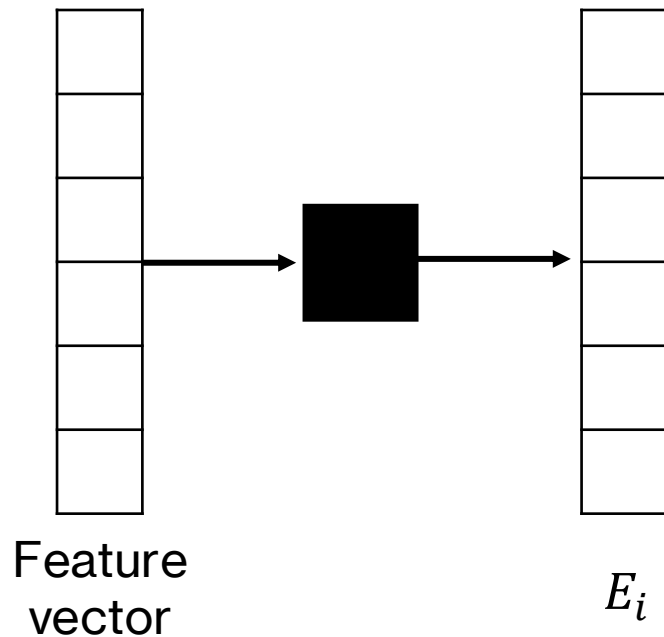
- Layer 1: central atom i learns about its immediate neighbors
- Layer 2: central atom i receives a message that now contains information about its neighbors' neighbors, generated in layer 1



Basics of Graph Neural Networks

Energy readout: the target

- Once message passing is done, each atom has a highly sophisticated context-aware feature vector
- The final vector is passed through a simple NN to predict local atomic energies

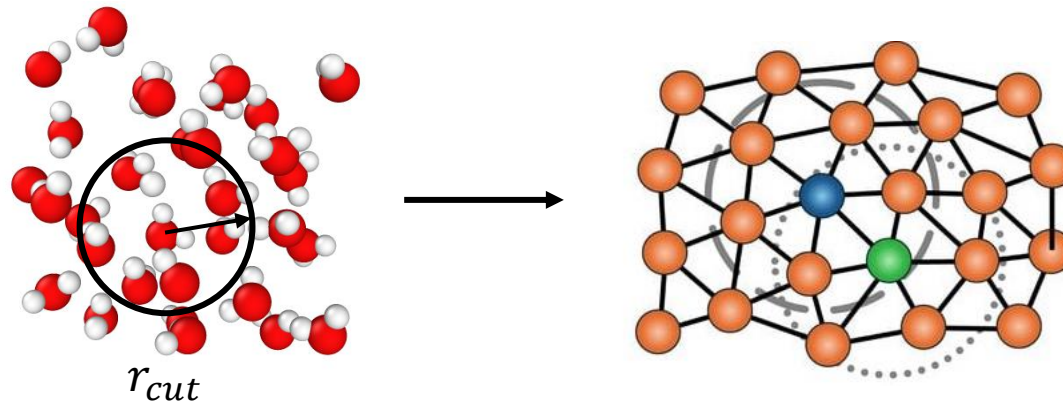


$$E_{tot} = \sum_i E_i$$

$$F_i = -\nabla_i E_{tot}$$

GNNs and MLIPs: the perfect match

- Atomic environments naturally map to graph descriptions
- No fixed bonds -> just interactions within the cutoff distance
- Energy is a sum of local atomic environments
 - You can train on small clusters and run simulations on much larger systems
 - Excellent size extrapolation



Classical FF vs. GNN MLIPs

Classical (empirical) FF

- Fixed equations
- Bond stretching term
- LJ parameters
- Manual parameterization

GNN MLIP

- Learned equations
- Bonding learned from data
- Interactions learned from data
- Automatic training

The neural network replaces the functional form of the forces – not Newton's equations

Agenda

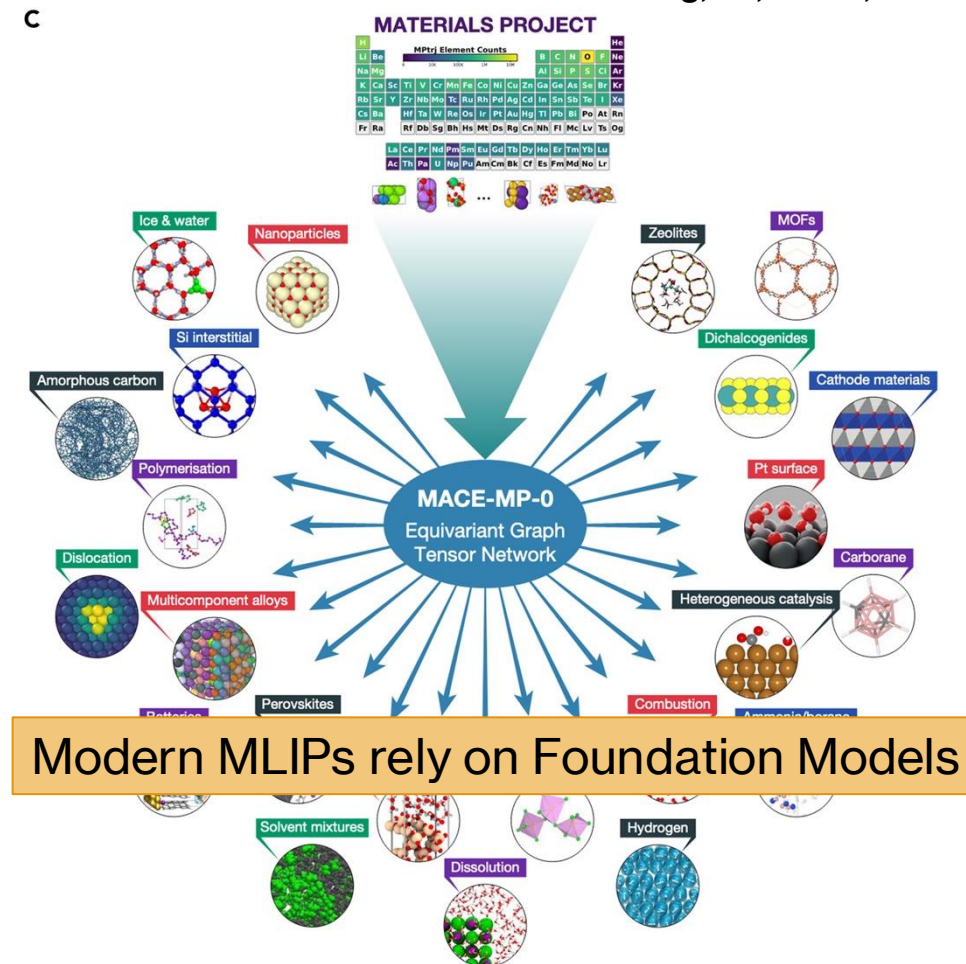
1. What are MLIPs and why are they useful?
2. Core concepts of MLIPs: representations, descriptors, architectures
3. Basics of Graph Neural Networks
- 4. Foundation models**
5. Data generation and training from scratch / finetuning
6. General considerations / best practices
7. The future of MLIPs

Foundation Models: The “Universal” potential

- Historically every group trained their own potential from self-generated DFT data

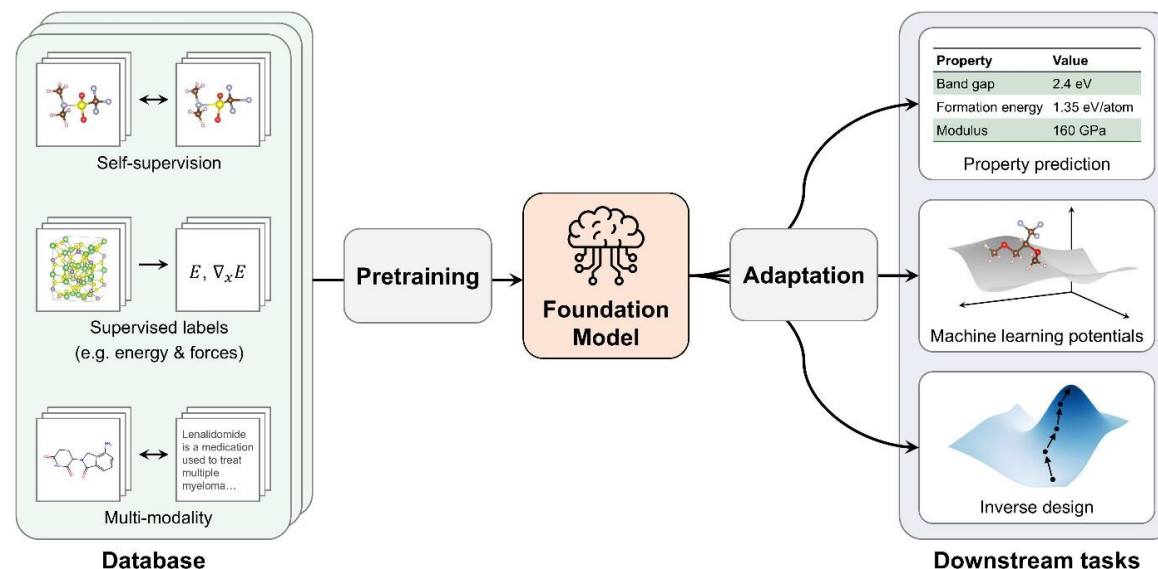
Wang, G., et. al., *iScience*, **2024**.

- Problems:
 - Expensive
 - System-specific
 - Months of work



Foundation Models: The “Universal” potential

- Shift from custom, single-system models to generalizable foundation models
- Models pre-trained on massive, diverse datasets containing millions of structure across the periodic table (e.g. the Materials Project)
- **Zero-shot capability** –model has learned the general “rules” of the periodic tables



Choi, J., et. al., *JACS Au.*, **2025**.

Foundation Models: The “Universal” potential

Popular Foundation Models (and their suggested applications)

Model	Architecture	Training Data	Best suited for
MACE-MPA0	Higher-order Equivariant GNN	Materials Project	Force predictions, organic-inorganic interfaces, complex reactions
MACE-OFF	Purely short-range, local equivariant GNN	Organic molecules at high levels of QM theory	Organic chemistry, drug-like molecules, biomolecules/proteins
MACE-POLAR	Polarizable Electrostatic GNN (MACE + long-range electrostatic induction solver)	OMol25 dataset (~100 million hybrid DFT calcs)	Systems with charge and spin states
CHGNet	Crystal GNN	Materials Project	Solid-state materials, phase stability
UMA	Mixture of Linear Experts (MoLE)	~500 mill unique 3D atomic structures spanning molecules, materials, and catalysts	Out-of-the-box predictions for catalysis, MOFs, molecules, and crystals

Foundation Models: The “Universal” potential

The Limits of Foundation Models

- Each model has a distinct boundary -> known for extrapolation failure

Where they shine:

- Near-equilibrium crystal structures, standard temperatures, bulk material properties

Where they struggle:

- Extreme conditions (high pressure/temp)
- Highly specific transition states in chemical reactions
- Systems with complex magnetic or electronic transitions

Agenda

1. What are MLIPs and why are they useful?
2. Core concepts of MLIPs: representations, descriptors, architectures
3. Basics of Graph Neural Networks
4. Foundation models
- 5. Data generation and training from scratch / finetuning**
6. General considerations / best practices
7. The future of MLIPs

Data Generation & Training

General Workflow for training

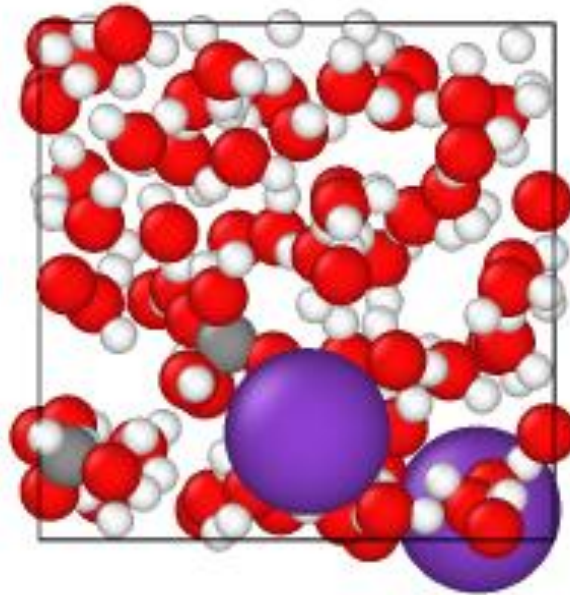
Generate Structures → **Run DFT** → **Train Model** → **Evaluate Performance**

- Run dynamics with a foundation model or previous trained model
- Randomly take snapshots
- Repeat at a variety of concentrations

- Choose level of theory

- Choose model architecture

- Validate the model's prediction against the DFT
- Validate the properties against experimental or other computational results

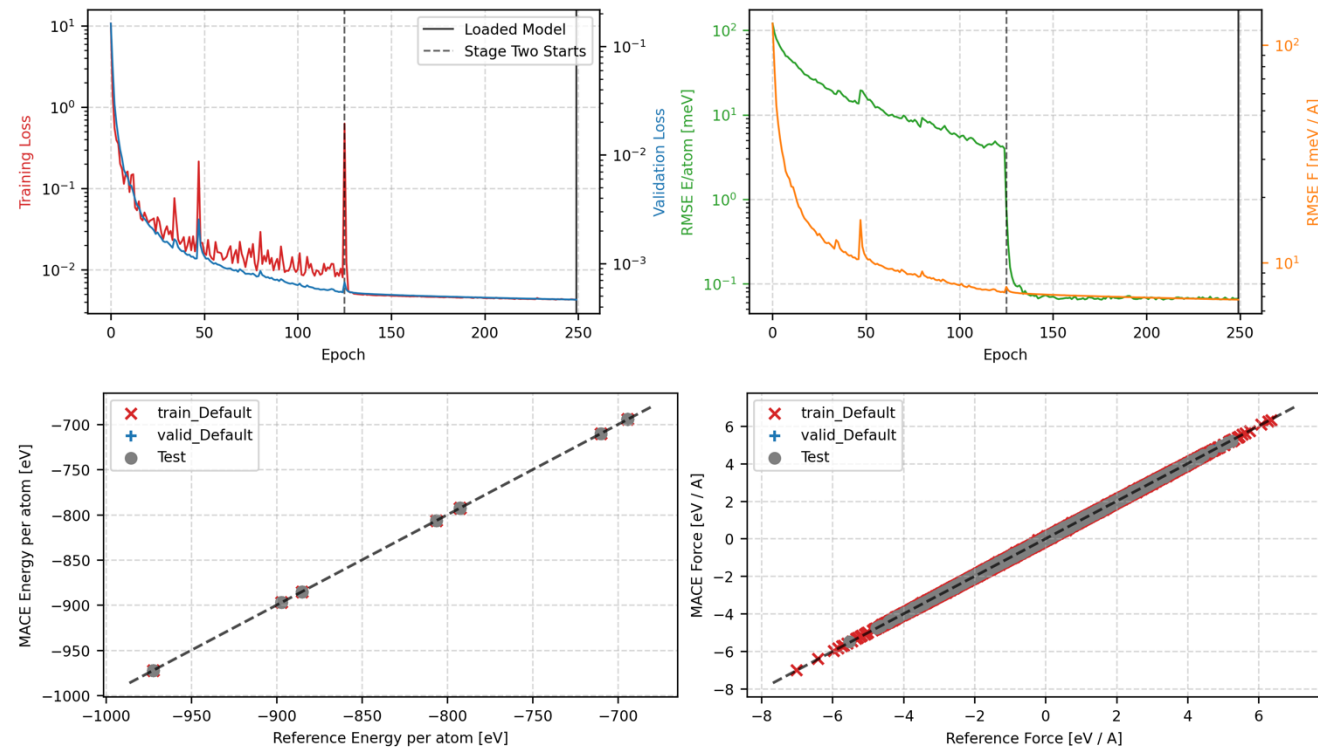


Model Validation

How to gauge the performance of your model

How well was the model trained?

Model loaded from epoch 249 (Default head)

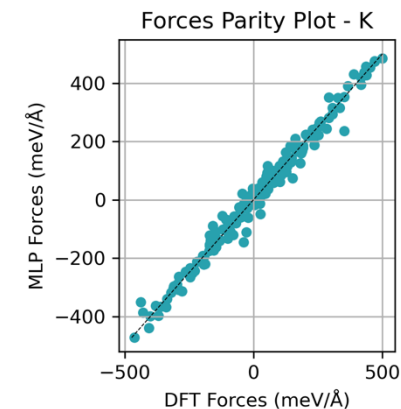
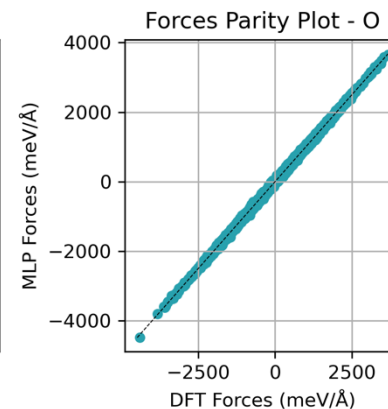
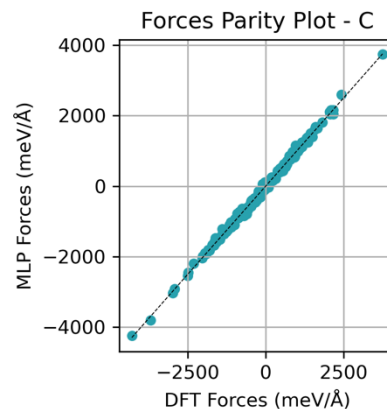
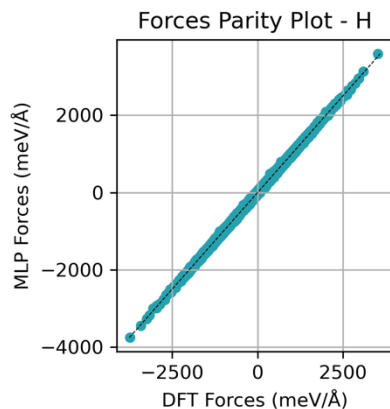
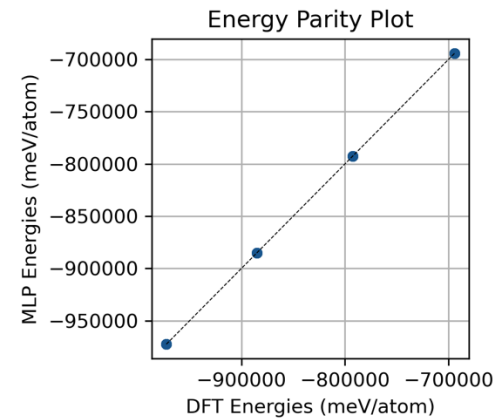


Model Validation

How to gauge the performance of your model

How well was the model trained?

- Model accurately reproduces DFT

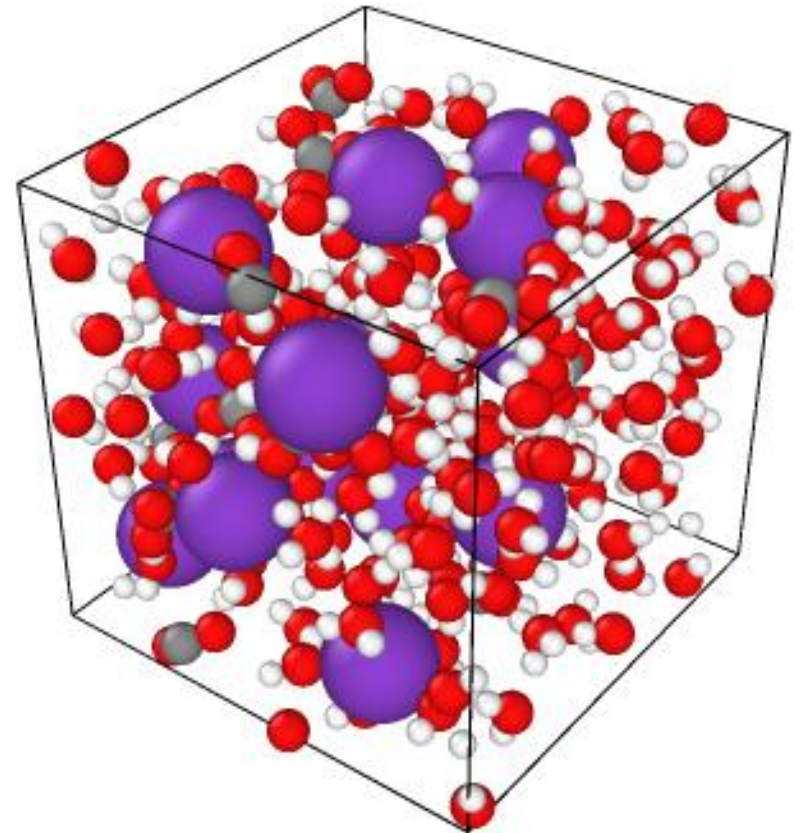
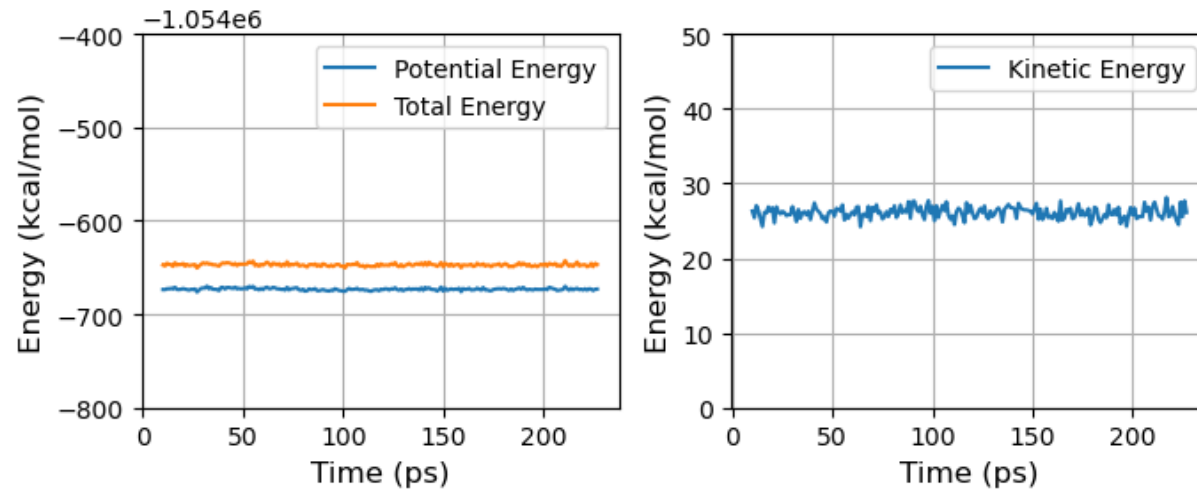


Model Validation

How to gauge the performance of your model

How well does it perform accurate dynamics?

- Are the dynamics stable?

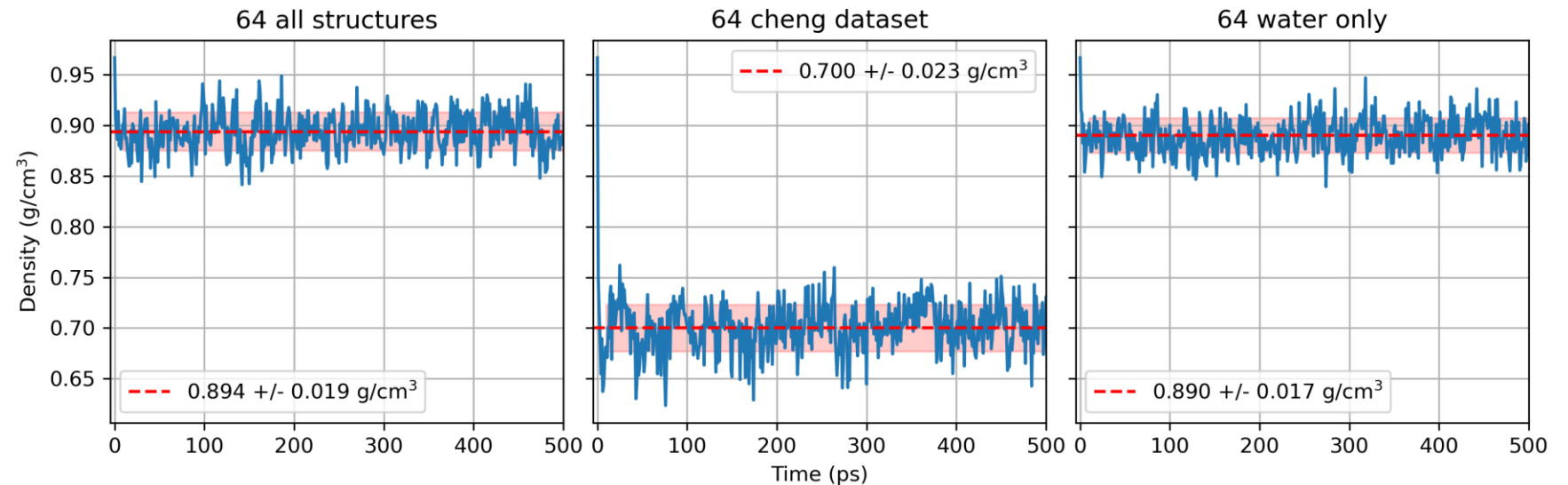


Model Validation

How to gauge the performance of your model

How well does it perform accurate dynamics?

- Are the dynamics stable?
- Does it reproduce experimental quantities?
 - Density

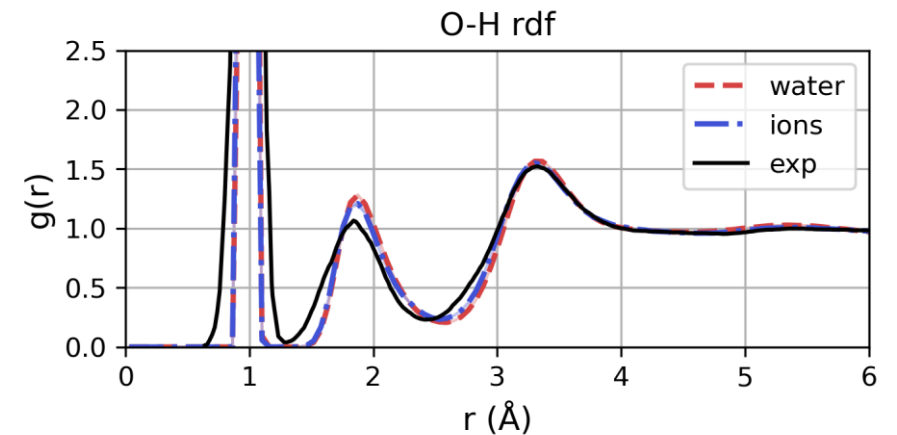
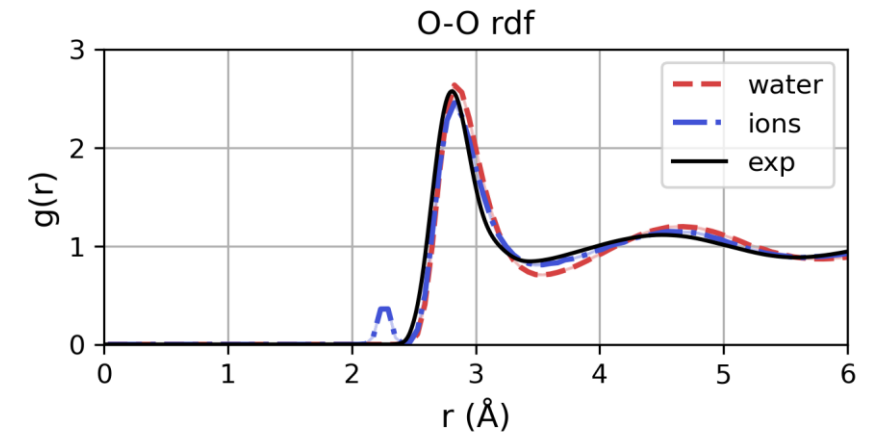


Model Validation

How to gauge the performance of your model

How well does it perform accurate dynamics?

- Are the dynamics stable?
- Does it reproduce experimental quantities?
 - Density
 - Structure factor / radial distribution functions

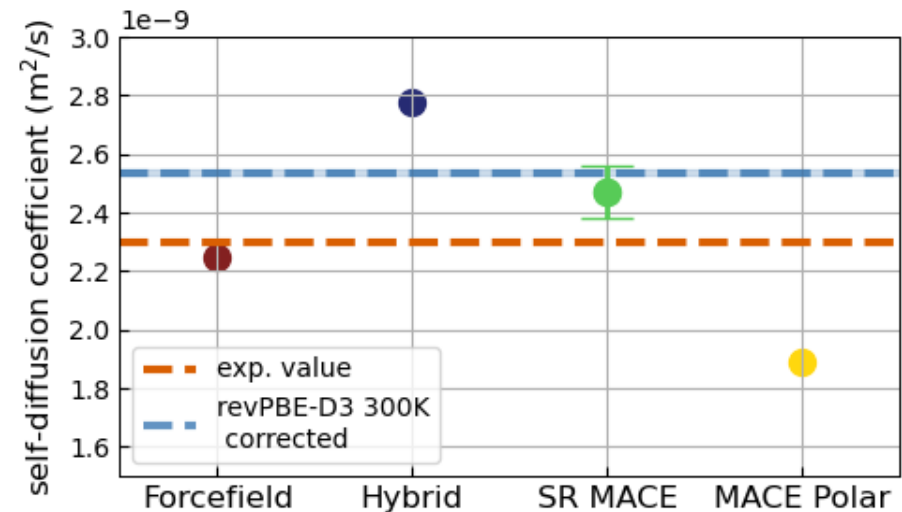
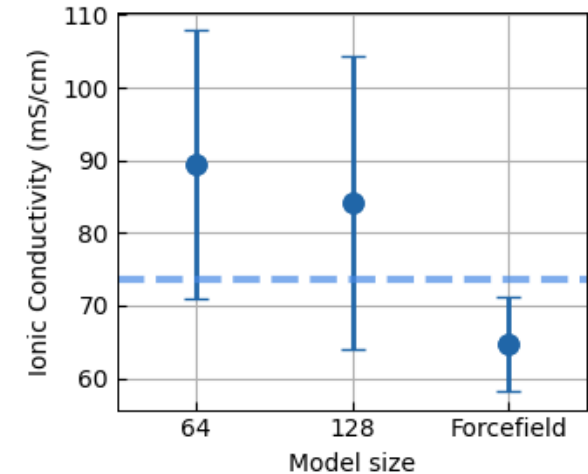


Model Validation

How to gauge the performance of your model

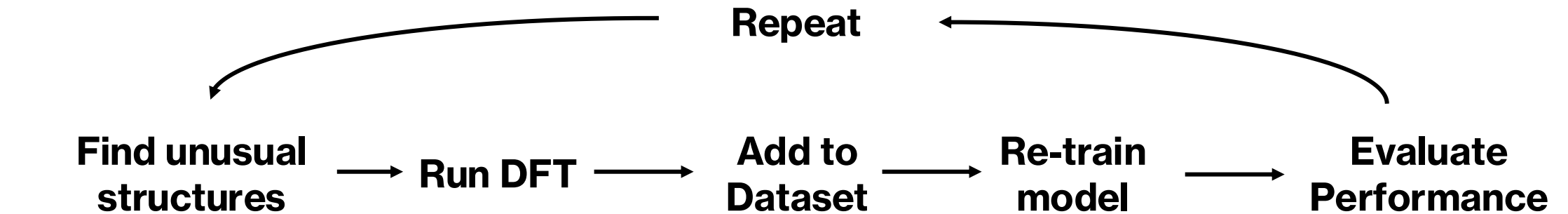
How well does it perform accurate dynamics?

- Are the dynamics stable?
- Does it reproduce experimental quantities?
 - Density
 - Structure factor / radial distribution functions
 - Conductivity / diffusion coefficients

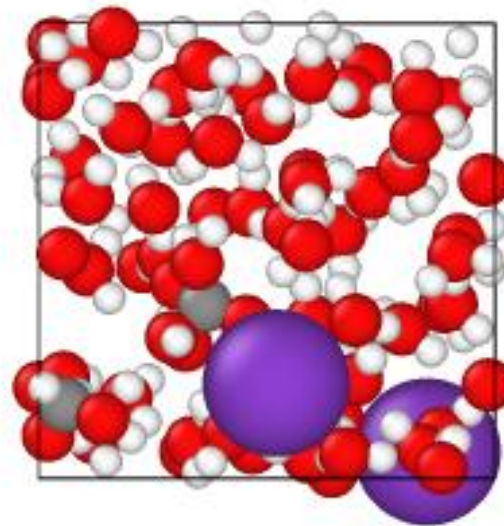


Data Generation & Training (in reality)

First pass through is normally insufficient performance



- Run dynamics with the trained model
- Extract “unusual” structure



Data Generation & Training: Build from Scratch or Fine-Tune?

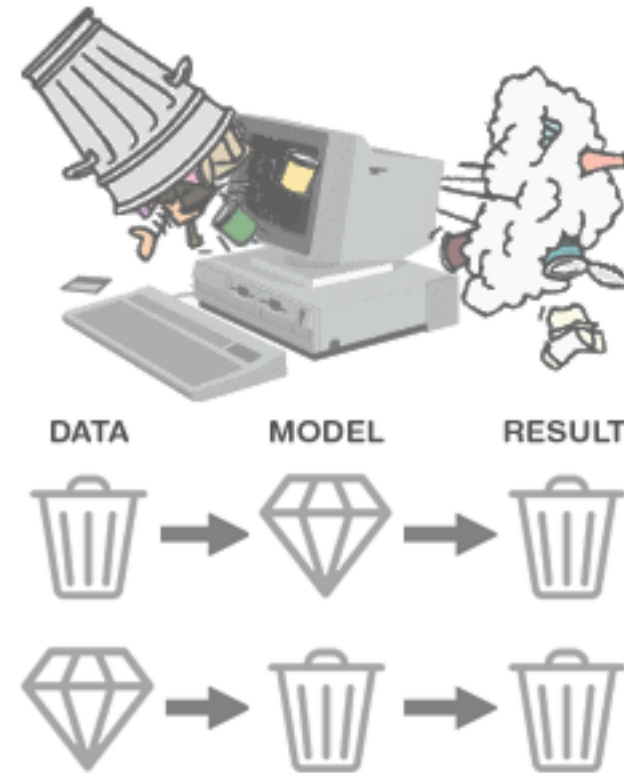


General rule of thumb: If a good foundation model already understands your chemistry, fine-tune it. Build from scratch only when existing models cannot describe your system.

Data Generation & Training

Garbage in, Garbage out

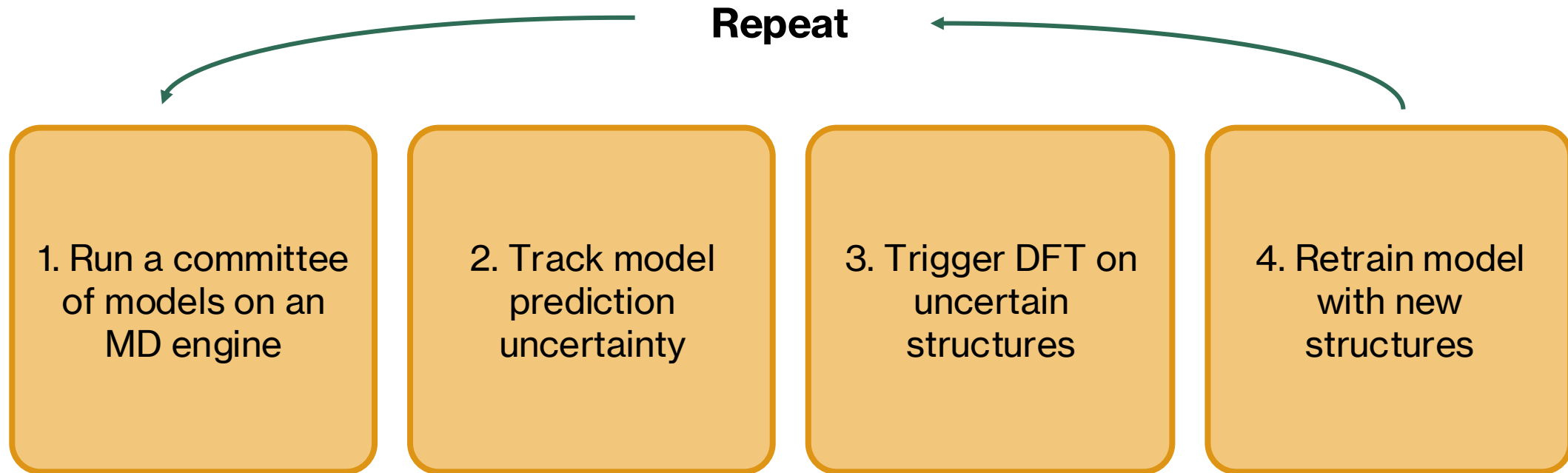
- Model only ever as good as the data it was trained on
- A model trained only on “relaxed” structures, the model is blind to atom vibrations and collisions



<https://www.vamsitalkstech.com/transformation/data-garbage-in-means-garbage-out-for-analytics/>

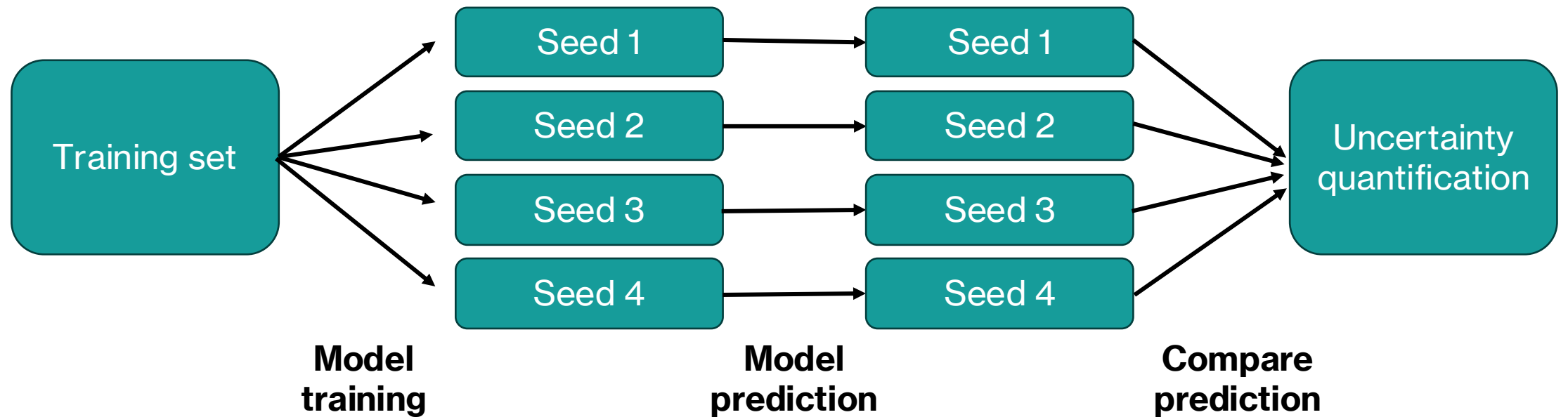
Data Generation & Training

Active Learning Loop



Uncertainty quantification

Model Committees



Uncertainty comes from the variance in the prediction from each different model trained on the SAME training set with the SAME hyperparameters.

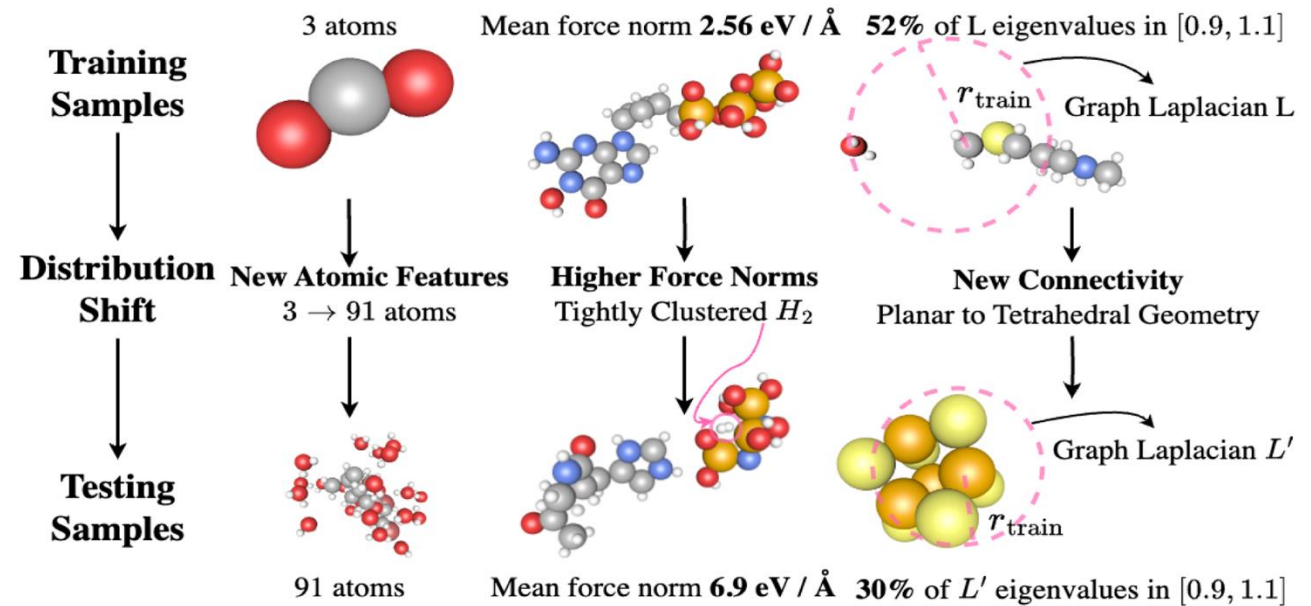
Agenda

1. What are MLIPs and why are they useful?
2. Core concepts of MLIPs: representations, descriptors, architectures
3. Basics of Graph Neural Networks
4. Foundation models
5. Data generation and training from scratch / finetuning
- 6. General considerations / best practices**
7. The future of MLIPs

Typical Failure Modes

Where to look if your model is performing poorly

- Outside training distribution
- Bad DFT reference data
- Insufficient sampling
- Wrong chemistry
- Long-range interactions
- Charge transfer



Kreiman, T. & Krishnapriyan A. S., *Digital Discovery*, **2026**.

General Considerations / Best Practices

Software Ecosystem

DFT: If training from scratch or adding data to fine-tune, you need to choose a software and functional for DFT

ML Models: most models have a github repo to clone the software

- MACE
 - When compiling your own software, be weary of the dependencies

MD Software:

- ASE – python-based molecular simulations. Most MLIPs have built-in ASE calculators
- LAMMPS – gold standard for scaling up MD simulations. Requires exporting a trained model using TorchScript and loading directly into GPU-enabled LAMMPS executable

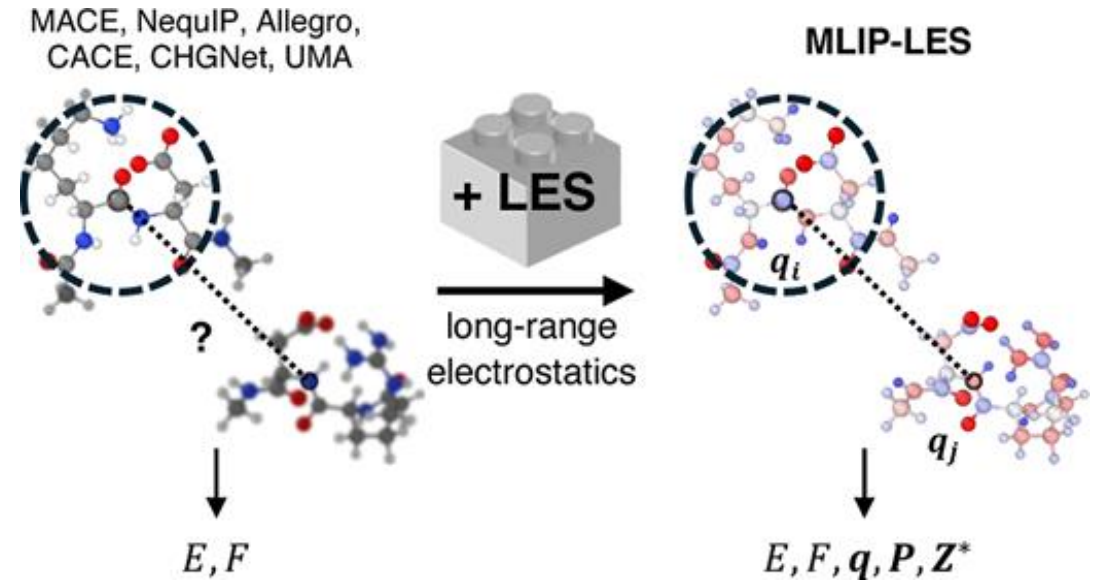
Agenda

1. What are MLIPs and why are they useful?
2. Core concepts of MLIPs: representations, descriptors, architectures
3. Basics of Graph Neural Networks
4. Foundation models
5. Data generation and training from scratch / finetuning
6. General considerations / best practices
- 7. The future of MLIPs**

Where is the field headed?

Beyond Local Interactions

- MLIPs are inherently short-range
- Long-range frameworks are being developed using different charge equilibration schemes or learning of “latent” charges



Where is the field headed?

Longer and larger simulation of any system

Development of the
scope of MLIPs

One force field per system

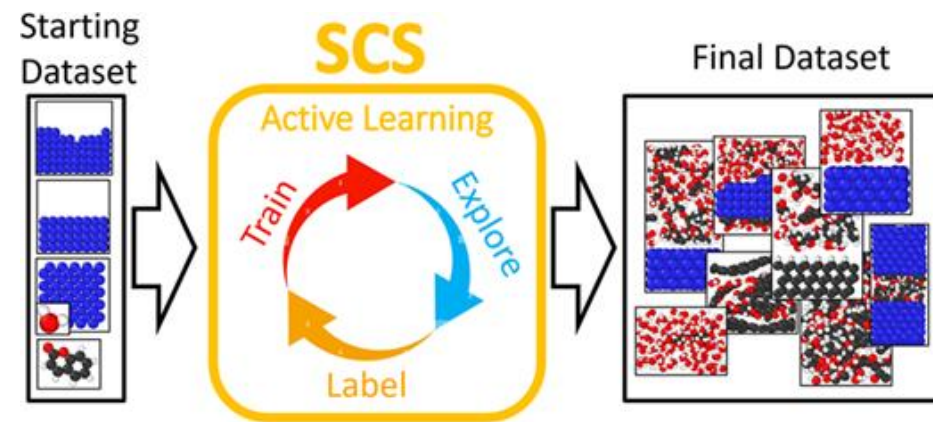


Foundation Model + fine-tuning



Autonomous, physics-aware,
universal atomistic simulators

Autonomous Data Generation



Additional References – comprehensive reviews

- **A Practical Guide to Machine Learning Interatomic Potentials** (*Ryan et al., 2025*)
- **Graph Neural Networks for Materials Science and Chemistry** (*Reiser et al., Communications Materials 2022*)
- **A Critical Review of ML Interatomic Potentials and Hamiltonian** (*Journal of Material Informatics, 2025*)
- **Reactive Machine Learning Interatomic Potentials** (*Chemical Reviews, 2026*)

Additional References – GNNs and Deep learning

- **"Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges"** (*Bronstein et al., 2021*)
- **"Neural Message Passing for Quantum Chemistry"** (*Gilmer et al., ICML 2017*)

Additional References – current state of the art models

- **NequIP** (*Batzner et al., Nature Communications 2022*)
- **Allegro** (*Musaelian et al., Nature Communications 2023*)
- **MACE** (*Batatia et al., NeurIPS 2022*)

Additional References – symmetry preservation

Invariant Atomic descriptors:

- **Atomic Cluster Expansion (ACE)** (Dusson et al., 2019).
- **Smooth Overlap of Atomic Positions (SOAP)** (Bartók et al., 2010).
- **Atom-Centered Symmetry Functions (ACSF)** (Behler & Parrinello, 2007).

Equivariant Deep Learning Architectures:

- **NequIP ($E(3)$ -Equivariant GNNs)** (Batzner et al., 2022).
- **MACE (Higher-Order Equivariant Message Passing)** (Batatia et al., 2022).